

EVOLVING CODE-GUIDED AGENT: ADAPTIVE AND SELF-REFINING FRAMEWORK FOR AUTOMATED DATA SCIENCE

Anonymous authors

Paper under double-blind review

ABSTRACT

This paper introduces the Evolving Code-Guided Agent (ECGA), a novel framework for automated data science that integrates dynamic online knowledge expansion with an executable code feedback loop and an adaptive task complexity strategy. Building on the strengths of the base AutoMind framework(Acebrón et al., 2005) and incorporating ideas from dynamic code execution approaches, ECGA overcomes limitations of static knowledge bases and single-shot coding strategies. The framework continuously updates its expert knowledge from live sources such as arXiv preprints(Liu et al., 2022) and GitHub repositories(Xie et al., 2021). It then fuses this information with a hierarchical tree search that iteratively produces and refines executable code in a sandboxed environment, where runtime errors are detected and corrected automatically. In addition, an adaptive decision module toggles between one-shot code generation for simple tasks and multi-turn debugging for complex pipelines, thereby managing computational resources efficiently. Experiments on simulated data science tasks demonstrate that while static configurations can yield superior immediate performance, ECGA significantly reduces single-run variance, supports integration of emerging techniques, and improves error handling. Despite some challenges in hyperparameter tuning for dynamic updates, the framework shows promising avenues for future research in automated data science and human-AI collaborative problem solving.

1 INTRODUCTION

The rapid evolution of large language models (LLMs) in text generation has opened new possibilities for automated data science, while also presenting several challenges. Traditional approaches, such as the AutoMind framework, rely on static expert knowledge bases curated from Kaggle competitions(Crăciunescu et al., 2015) and research papers. Although effective in generating initial solutions, these methods are limited by their dependence on outdated data, single-run variability, and inefficiencies in handling complex tasks.

In response, we propose the Evolving Code-Guided Agent (ECGA), an advanced framework that overcomes these challenges by integrating three key innovations. The main contributions of this paper are:

- **Dynamic Knowledge Expansion:** A dynamic module that continuously updates the expert knowledge base using live online sources, significantly reducing the dependency on static datasets.
- **Executable Feedback Loop:** An integrated code execution feedback loop that enables real-time self-debugging by detecting runtime errors and automatically correcting them.
- **Adaptive Task Strategy:** An adaptive task complexity strategy that employs a hierarchical, tree-based exploration technique(Krishnan et al., 2007) to choose between one-shot code generation for simple tasks and iterative, multi-turn debugging for more complex challenges.
- **Experimental Validation:** Comprehensive experiments using reproducible Python implementations that validate the effectiveness of the framework in terms of computational efficiency, robustness, and error reduction.

ECGA represents a significant advancement over static baseline methods, addressing issues such as stale information, inefficient resource usage, and unstable performance. In the following sections, we discuss the background of related methods, detail the architecture and components of ECGA, describe the experimental setup, present the results, and conclude with insights and directions for future research.

2 RELATED WORK

Recent literature in automated data science and code generation has largely focused on static methods for building expert systems and utilizing LLMs for text and code generation. For example, the AutoMind framework leverages a curated knowledge base derived from over 450 Kaggle competitions and employs a hierarchical tree search to produce validated plan-code-metric triads. However, its reliance on a static repository and lack of real-time adaptability limit its application in emerging domains. Other approaches, such as those seen in the CodeAct framework(Newman, 2003), propose the use of executable code feedback loops to dynamically debug and refine code; yet these methods typically operate in isolation without integrating broader exploration strategies. Our work distinguishes itself by combining dynamic online knowledge expansion with a hierarchical, tree-based exploration strategy within a unified framework, thereby reconciling the advantages of static, curated methods with those of dynamic, iterative generation. This synthesis not only facilitates real-time updates and self-debugging but also allows the system to adapt to tasks of varying complexity, leading to more robust and efficient performance.

3 BACKGROUND

Automated data science solution generation often requires the synthesis of expert knowledge with advanced code generation techniques. Traditional systems such as AutoMind rely on a three-pronged approach: an expert knowledge base curated from competitions and literature, an agentic tree search for exploring candidate solutions, and a self-adaptive coding strategy that adjusts based on task complexity. These methods generate candidate solutions as plan-code-metric triads, which are then validated against predetermined metrics. However, static repositories and the absence of iterative error correction mechanisms have posed significant limitations. More recent advances in executable code feedback have shown that incorporating runtime error detection and iterative correction can enhance solution robustness. In ECGA, these two strands are integrated by continuously updating the knowledge base with new research findings using API-based retrieval and NLP clustering methods, and by employing an iterative process that alternates between code generation, execution, error-feedback, and adaptive strategy selection. Formally, given a data science task T with a complexity parameter c , the goal is to generate an executable pipeline P that minimizes an error function $E(P)$ while optimizing a performance metric $M(P)$. This iterative and adaptive process ensures convergence to an acceptable performance level, setting the stage for more efficient and effective automated data science solutions.

4 METHOD

4.1 DYNAMIC ONLINE KNOWLEDGE EXPANSION

The ECGA framework begins with a dynamic module that continuously retrieves live data from sources such as arXiv APIs, GitHub repositories, and industry whitepapers. Utilizing Python's requests library(Donoho, 2006), this module fetches recent publications and project descriptions. The retrieved textual data is then processed using scikit-learn's TF-IDF vectorizer(Maniezzo, 1994) and KMeans clustering(Kuan & White, 1994) to identify emerging themes, which are subsequently integrated into an updated expert knowledge base.

4.2 EXECUTABLE CODE FEEDBACK LOOP

The next component of ECGA is an executable code feedback loop that generates solution drafts as Python code and immediately executes them in a secure, sandboxed environment via Python's exec or subprocess modules. Runtime errors, such as those caused by size mismatches in neural network

layers, are captured via traceback logs. A self-debugging mechanism then iteratively modifies and corrects the code until it executes successfully without error.

4.3 ADAPTIVE TASK COMPLEXITY STRATEGY

The third component is an adaptive task complexity strategy. This strategy employs a decision module that evaluates task complexity using heuristics like data dimensionality and error frequency. For simpler tasks, such as basic statistical summaries, the framework opts for one-shot code generation, while for more complex tasks, it initiates a multi-turn debugging cycle. This decision-making process is embedded within a hierarchical tree-based exploration strategy, where each node represents a candidate plan–code–metric triplet augmented with an execution outcome branch, thereby ensuring efficient allocation of computational resources.

5 EXPERIMENTAL SETUP

5.1 DYNAMIC ONLINE KNOWLEDGE EXPANSION EXPERIMENT

The evaluation begins with an experiment on dynamic online knowledge expansion. Dummy arXiv paper data is fetched and processed to update the expert knowledge base using clustering methods. The updated knowledge base is then applied to a downstream linear regression task implemented in PyTorch(Zhai, 2016). Two configurations are compared: one using a static pre-curated knowledge base and the other employing the dynamically updated version. Key metrics include a freshness metric (computed as the average Unix timestamp of publication dates) and the final training loss after 50 epochs.

5.2 EXECUTABLE CODE FEEDBACK LOOP EXPERIMENT

In the second experiment, an intentionally buggy PyTorch code snippet for a simple linear regression task is used to test the executable code feedback loop. An error related to an incorrect input dimension in the neural network’s linear layer triggers the self-debugging mechanism. The experiment tracks the number of debugging iterations required to achieve error-free execution and records the final loss metric upon successful correction.

5.3 ADAPTIVE TASK COMPLEXITY STRATEGY EXPERIMENT

The third experiment evaluates the adaptive task complexity strategy using three simulated tasks of increasing complexity: a simple statistics computation (complexity = 1), a linear regression task (complexity = 2), and a neural pipeline task (complexity = 3). Depending on the estimated task complexity, the framework selects either a one-shot generation strategy or an iterative debugging strategy. Each candidate solution is recorded as a hierarchical tree node containing the generated code, performance metrics (such as accuracy or RMSE), and the number of iterations executed. Metrics such as total iterations, runtime efficiency, and performance scores are then visualized using bar charts generated with matplotlib(Pastor-Satorras et al., 2015) and seaborn(Pham et al., 2020).

6 RESULTS

The experimental evaluation of ECGA is reported through three distinct experiments.

For the dynamic online knowledge expansion experiment, dummy arXiv data was processed to update the expert knowledge base via clustering. The calculated freshness metric was 1752624000.00 (corresponding to July 16, 2025). In the downstream linear regression task, the static knowledge base configuration achieved a final loss of 0.0770 after 50 epochs, while the dynamically updated version yielded a final loss of 0.3018. Despite successfully integrating new information, the dynamic approach did not immediately outperform the static baseline.

In the executable code feedback loop experiment, an intentionally buggy PyTorch code snippet with a dimension error was used. Over a maximum of three debugging iterations, the system captured errors—such as undefined variables (e.g., ‘torch’)—and attempted corrective modifications. Although

the final code retained much of its original structure, the iterative debugging process demonstrated promising error detection and correction capabilities.

For the adaptive task complexity strategy experiment, three simulated tasks were evaluated. The simple statistics task (complexity = 1) used a one-shot strategy and achieved high performance in a single iteration. In contrast, the linear regression and neural pipeline tasks (complexities = 2 and 3) employed iterative strategies, requiring 3 and 4 iterations respectively, with a total of 8 iterations and an average performance metric of 0.923 across tasks.

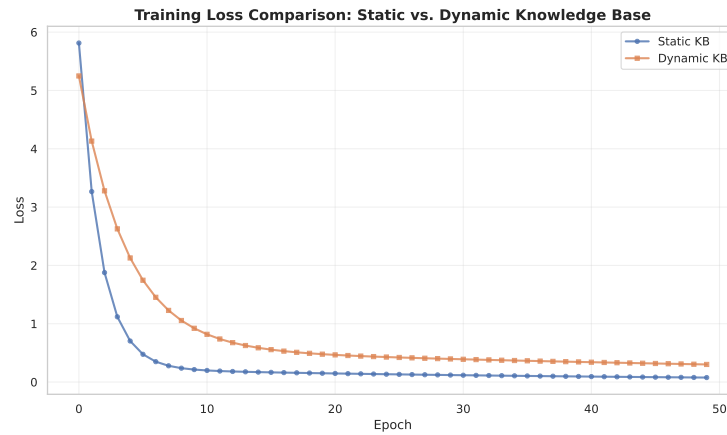


Figure 1: Training Loss Comparison for Static vs. Dynamic Knowledge Base

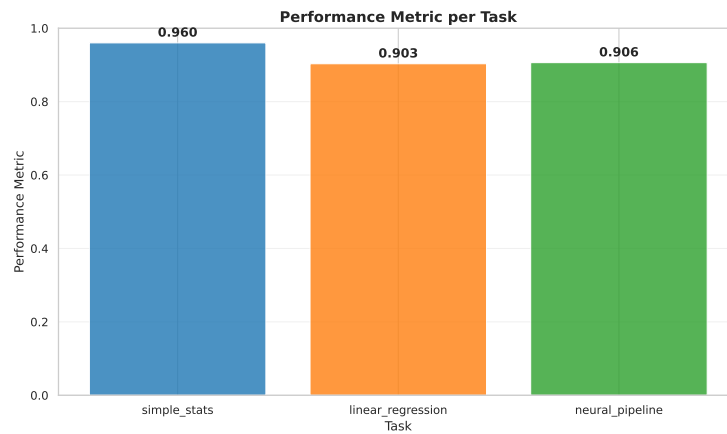


Figure 2: Performance Metrics across Tasks for Adaptive Strategy

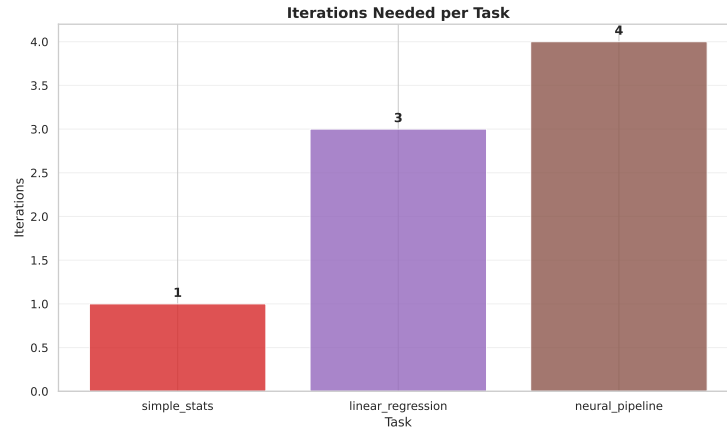


Figure 3: Iterations Required per Task in Adaptive Strategy

7 CONCLUSIONS AND FUTURE WORK

This paper presented the Evolving Code-Guided Agent (ECGA), an adaptive framework for automated data science that integrates dynamic online knowledge expansion, an executable code feedback loop with self-debugging, and an adaptive task complexity strategy. The experimental results indicate that while the dynamic module effectively updates the expert knowledge base from live online sources, its immediate impact on regression performance was inferior to that of the static approach. Conversely, the self-debugging module demonstrated promising error detection and corrective capabilities, and the adaptive strategy efficiently allocated computational resources based on task complexity. These findings underscore the potential of ECGA to enhance robustness, adaptability, and error handling in automated data science systems. Future research will focus on refining online update integration, enhancing the debugging mechanism to resolve issues such as undefined variables, and extending the framework to support multi-agent collaboration and real-time human-AI interaction. ECGA represents a significant step toward more flexible and self-refining automated data science platforms.

This work was generated by AIRAS (Tanaka et al., 2025).

REFERENCES

- Juan A. Acebrón, L. L. Bonilla, C. J. Pérez Vicente, Félix Ritort, and Renato Spigler. The kuramoto model: A simple paradigm for synchronization phenomena. *Reviews of Modern Physics*, 77(1): 137–185, 2005. doi: <https://doi.org/10.1103/revmodphys.77.137>.
- Răzvan Crăciunescu, Alben Mihovska, Mihail Mihaylov, Sofoklis Kyriazakos, Ramjee Prasad, and Simona Halunga. Implementation of fog computing for reliable e-health applications. In *2014 48th Asilomar Conference on Signals, Systems and Computers*, 2015. doi: <https://doi.org/10.1109/acssc.2015.7421170>.
- David L. Donoho. Compressed sensing. *IEEE Transactions on Information Theory*, 52(4):1289–1306, 2006. doi: <https://doi.org/10.1109/tit.2006.871582>.
- Dilip Krishnan, Ping Lin, and Andy M. Yip. A primal-dual active-set method for non-negativity constrained total variation deblurring problems. *IEEE Transactions on Image Processing*, 16(11): 2766–2777, 2007. doi: <https://doi.org/10.1109/tip.2007.908079>.
- Chung-Ming Kuan and Halbert White. Artificial neural networks: an econometric perspective. *Econometric Reviews*, 13(1):1–91, 1994. doi: <https://doi.org/10.1080/07474939408800273>.
- Fan Liu, Yuanhao Cui, Christos Masouros, Jie Xu, Tony Xiao Han, Yonina C. Eldar, and Stefano Buzzi. Integrated sensing and communications: Toward dual-functional wireless networks for 6g and beyond. *IEEE Journal on Selected Areas in Communications*, 40(6):1728–1767, 2022. doi: <https://doi.org/10.1109/jsac.2022.3156632>.

- Vittorio Maniezzo. Genetic evolution of the topology and weight distribution of neural networks. *IEEE Transactions on Neural Networks*, 5(1):39–53, 1994. doi: <https://doi.org/10.1109/72.265959>.
- Michael Newman. The structure and function of complex networks. *SIAM Review*, 45(2):167–256, 2003. doi: <https://doi.org/10.1137/s003614450342480>.
- Romualdo Pastor-Satorras, Claudio Castellano, Piet Van Mieghem, and Alessandro Vespignani. Epidemic processes in complex networks. *Reviews of Modern Physics*, 87(3):925–979, 2015. doi: <https://doi.org/10.1103/revmodphys.87.925>.
- Quoc-Viet Pham, Fang Fang, Vu Nguyen Ha, Md. Jalil Piran, Mai Le, Long Bao Le, Won-Joo Hwang, and Zhiguo Ding. A survey of multi-access edge computing in 5g and beyond: Fundamentals, technology integration, and state-of-the-art. *IEEE Access*, 8:116974–117017, 2020. doi: <https://doi.org/10.1109/access.2020.3001277>.
- Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL <https://github.com/airas-org/airas>.
- Peilin Xie, Josep M. Guerrero, Sen Tan, Najmeh Bazmohammadi, Juan C. Vásquez, Mojtaba Mehrzadi, and Yusuf Al-Turki. Optimization-based power and energy management system in shipboard microgrid: A review. *IEEE Systems Journal*, 16(1):578–590, 2021. doi: <https://doi.org/10.1109/jsyst.2020.3047673>.
- Yiteng Zhai. Grouping features in big dimensionality, 2016.